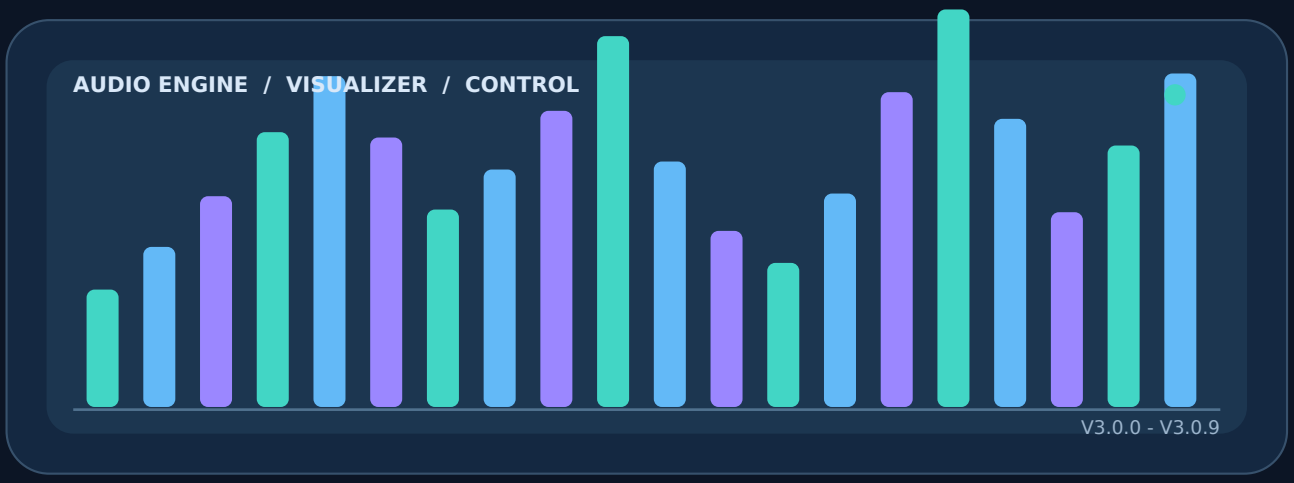
PROJECT TECHNICAL DOSSIER / BILINGUAL IMPLEMENTATION REPORT

FLOATING DJ VISUALIZER

V3.0.0 - V3.0.9

Equalizer Engine Integration & Audio Architecture



DOCUMENT SCOPE	BASELINE / FINAL	PLATFORM PROFILE
Implementation additions across the Version 3 milestone	V3.0.0 → V3.0.9 versionCode 70 → 81	minSdk 26 · targetSdk 36 Test handset: Redmi 10C / Android 11 (API 30)

Prepared for project documentation, developer handover, and technical review.
Disusun sebagai dokumentasi implementasi untuk pemahaman teknis, serah-terima pengembangan, dan peninjauan arsitektur.

Developer: Rifki • Project: Floating DJ Visualizer • Document revision: 1.0

01 Executive Summary | Ringkasan Eksekutif

What shipped, what the code proves, and what remains device-dependent. / Apa yang dikirim, apa yang dibuktikan source, dan apa yang tetap bergantung perangkat.

EN. Version 3.0.0–3.0.9 adds a modular experimental Equalizer Engine to an established floating audio visualizer. The implementation includes a dedicated EQ Activity, runtime effect discovery, a session-0 controller, an API-28+ DynamicsProcessing 10-band path, a native Equalizer fallback, optional BassBoost and Virtualizer effects, Input Gain (Preamp), a DynamicsProcessing Limiter stage, preset persistence/import/export, Theme Studio integration, and notification controls.

ID. Versi 3.0.0–3.0.9 menambahkan Equalizer Engine eksperimental yang modular ke aplikasi floating audio visualizer. Implementasi mencakup Activity EQ terpisah, pencarian effect saat runtime, controller session-0, jalur DynamicsProcessing 10-band pada API 28+, fallback Equalizer native, efek BassBoost dan Virtualizer opsional, Input Gain (Preamp), stage Limiter DynamicsProcessing, penyimpanan/import/export preset, integrasi Theme Studio, serta kontrol notifikasi.

Area	Implementation status	Evidence / note
10-band EQ	Implemented in source; user-confirmed on test handset	DynamicsProcessing config + band read-back; output-mix scope experimental.
5-band / native fallback	Implemented	Uses `Equalizer.numberOfBands`, center frequencies and level range reported by the active effect.
Bass Boost / Virtualizer	Implemented as optional session-0 effects	Creation/control and strength read-back are checked; availability depends on vendor.
Preamp	Implemented for DynamicsProcessing backend	Uses Input Gain stage; range -12 dB to +6 dB; fallback backend reports unavailable.
Limiter	Implemented as DynamicsProcessing stage	In-use stage, default bypassed; ON/OFF; threshold -1 dBFS, ratio 20:1, attack 5 ms, release 100 ms.
Reverb	Detection only; UI intentionally disabled	Descriptor inventory reports auxiliary/post-processing capability; no external player attach/send integration.
Global processing	Not guaranteed	Session 0 is experimental/deprecated for insert effects; route coverage must not be overstated.

CORE ENGINEERING POSITION / POSISI TEKNIS UTAMA

The application reports an effect as active only when the selected backend has control and is enabled. A successful constructor, descriptor, or UI toggle is not proof that every third-party player or output route is processed. / Aplikasi melaporkan engine aktif hanya ketika backend terpilih memiliki kontrol dan status enabled. Constructor, descriptor, atau toggle UI yang berhasil bukan bukti bahwa semua player pihak ketiga atau seluruh route audio diproses.

02 Version 3 Milestone History | Riwayat Milestone Versi 3

A consolidated progression from the initial modular EQ to the v3.0.9 baseline. / Rangkaian perkembangan dari EQ modular awal hingga baseline v3.0.9.



Milestone grouping is based on preserved project artifacts and documented development outcomes; it is not a claim that every internal patch had a separate public release.

Milestone	Engineering additions	User-facing outcome
V3.0.0	Dedicated EqualizerActivity; EffectCapabilityProbe; EqEngineController; EqEngineService; EqProfile; EqPresetRepository; EqAudioRouteMonitor; initial native Equalizer/BassBoost/Virtualizer session-0 attempt.	EQ entry point and first experimental effect path, preset model, device inventory.
V3.0.1	Adaptive backend selection: DynamicsProcessing 10-band on API 28+ when constructible/controllable; native Equalizer fallback. Current working-state persistence and service restoration strengthened.	10 bands on the tested handset; actual native band count otherwise; values preserved across Activity close/reopen.
V3.0.2	DynamicsProcessing Limiter stage reserved and controllable; clearer Current State vs saved preset tracking.	Limiter ON/OFF and more explicit working-state status.
V3.0.3	Input Gain / Preamp control; explicit Reverb descriptor capability reporting; empty-state message when EQ is off.	Preamp control where supported; Reverb is reported but remains disabled without attach/send path.
V3.0.5	Equalizer UI connected to Theme Studio colors, language and text scale; EQ entry card status integrated with MainActivity.	Consistent Glass/Light/Dark/Custom/System theming and main-page EQ status.
V3.0.6 - 3.0.7	Mixer interaction, preset workflow, direct dB entry, individual reset and horizontal 5-of-10 band canvas iterations.	More precise control and clearer preset management.
V3.0.8 - 3.0.8.1	Notification actions for EQ and visualizer; Start/Stop Visualizer wording; notification drawable resource correction.	Quick actions from the foreground notification; compile fix for app-owned stop icon.
V3.0.9	Precision custom MixerFaderView, directional touch handling, horizontal band navigation, 5/10-band starter presets and version metadata finalized.	Improved fader interaction and v3.0.9 installation baseline.

Version notes / Catatan versi

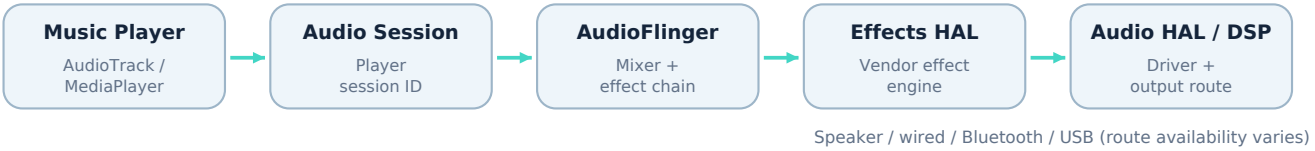
The milestone list consolidates the surviving v3.0.0 and v3.0.9 project artifacts plus the recorded implementation iterations. It is not a claim that every intermediate patch was published as a separate GitHub release. Version 3.0.4 is not separately described because no independent artifact or reliable change record was supplied for it.

Daftar milestone ini merangkum artefak project v3.0.0 dan v3.0.9 yang tersedia serta catatan iterasi implementasi. Daftar ini bukan klaim bahwa setiap patch perantara diterbitkan sebagai GitHub Release tersendiri. Versi 3.0.4 tidak dijelaskan terpisah karena tidak tersedia artefak atau catatan perubahan independen yang dapat diverifikasi.

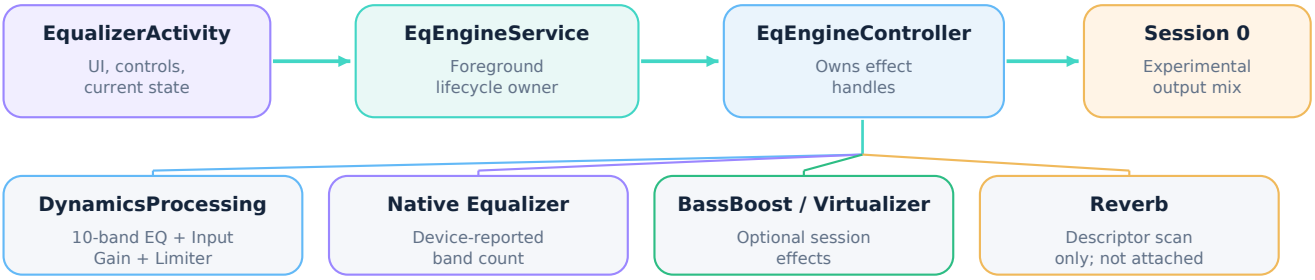
03 System Architecture | Arsitektur Sistem

Separation of visualization, UI, effect ownership, preset data and Android audio plumbing. / Pemisahan visualizer, UI, pemilik effect, data preset, dan jalur audio Android.

PLAYBACK / OUTPUT PIPELINE



FLOATING DJ V3 EQ EXPERIMENT - IMPLEMENTED SESSION-0 PATH



Separate visualization tap: FloatingVisualizerService -> Visualizer(0) -> FFT -> preview / floating renderer

EN. The visualizer and the EQ are related to audio but are separate processing responsibilities. `FloatingVisualizerService` obtains audio visualization data through the Android `Visualizer` path and feeds FFT/spectrum rendering. The EQ path is owned by `EqEngineService` and `EqEngineController`; it creates and controls audio effects. The UI Activity sends parameter changes and reads state but does not own the long-lived native effect handles.

ID. Visualizer dan EQ sama-sama berhubungan dengan audio, tetapi tanggung jawab pemrosesannya dipisahkan. `FloatingVisualizerService` memperoleh data visualisasi melalui jalur Android `Visualizer` untuk FFT/spektrum. Jalur EQ dikelola oleh `EqEngineService` dan `EqEngineController` untuk membuat serta mengendalikan audio effect. Activity UI mengirim perubahan parameter dan membaca status, tetapi bukan pemilik handle native jangka panjang.

Layer / component	Responsibility	Key files
MainActivity	EQ entry card, Theme Studio link, traffic-light status, notification host integration.	MainActivity.kt; activity_main.xml
EqualizerActivity	EQ screen, band canvas, fader controls, effect toggles, presets/import/export, theme/language/scale.	EqualizerActivity.kt
EffectCapabilityProbe	Read-only `AudioEffect.queryEffects()` inventory and Reverb descriptor filtering.	eq/core/EffectCapabilityProbe.kt
EqEngineController	Owns backend selection and native effect handles; applies bands and effect parameters; reports read-back/control.	eq/core/EqEngineController.kt
EqEngineService	Foreground lifecycle owner; starts/stops engine, restores saved state, refreshes notification.	eq/service/EqEngineService.kt
EqPresetRepository / EqProfile	Profile model, SharedPreferences persistence, JSON schema and preset CRUD.	eq/data/EqPresetRepository.kt; eq/model/EqProfile.kt
EqAudioRouteMonitor	Lists connected output device types exposed through AudioManager.	eq/route/EqAudioRouteMonitor.kt
FloatingVisualizerService	FFT, visualization rendering, floating overlay, media/notification controls.	FloatingVisualizerService.kt

04 Audio Routing & Session Scope | Jalur Audio dan Cakupan Session

What session 0 means, what is actually controlled, and why this is not a kernel-level EQ. / Makna session 0, apa yang dikendalikan, dan mengapa ini bukan EQ tingkat kernel.

Android playback route (conceptual). A player such as VLC creates an `AudioTrack` or `MediaPlayer` with an audio session. Android's media framework / AudioFlinger mixes streams and applies effect chains. Effects HAL and Audio HAL connect framework processing with vendor implementations and physical output routes such as speaker, wired headset, Bluetooth or USB audio.

Jalur playback Android (konseptual). Player seperti VLC membuat `AudioTrack` atau `MediaPlayer` dengan audio session. Framework media Android / AudioFlinger mencampur stream dan menerapkan effect chain. Effects HAL dan Audio HAL menghubungkan pemrosesan framework dengan implementasi vendor serta output fisik seperti speaker, headset kabel, Bluetooth, atau USB audio.

Approach	What the project does	Scope / limitation
DynamicsProcessing(0, 0, config)	Creates a 10-band DynamicsProcessing instance on session ID 0 when API >= 28; accepts it only when `hasControl()` and channelCount checks pass.	Experimental output-mix session. It does not prove every application's stream or every physical route is processed.
Equalizer(0, 0)	Fallback native Equalizer on session 0; enumerates actual number of bands, center frequency and band level range.	Legacy insert effect. Session-0 insert use is deprecated in Android API docs and implementation varies by device.
BassBoost(0, 0) / Virtualizer(0, 0)	Optional effect handles; only retained when control is obtained; strength/enable queried and read back.	Support and ownership can vary; another audio effect can take control.
Reverb descriptor scan	Enumerates descriptors and identifies Reverb `connectMode` entries; UI explains attachment requirement and keeps toggle disabled.	No external player session ID is injected, and no auxiliary send level is set by this app.

SESSION 0 IS NOT A UNIVERSAL GLOBAL AUDIO API

The code uses session ID 0 for an output-mix experiment. Android documentation deprecates insert EQ/BassBoost/Virtualizer use on the global output mix. A successful `hasControl()` is useful evidence for the tested device state, but not a cross-app or cross-route guarantee. / Kode menggunakan session ID 0 sebagai eksperimen output-mix. Dokumentasi Android menandai penggunaan insert EQ/BassBoost/Virtualizer pada global output mix sebagai deprecated. `hasControl()` yang berhasil adalah bukti berguna untuk kondisi perangkat yang diuji, bukan jaminan semua aplikasi atau route.

No direct kernel / HAL writes. The APK does not modify firmware, kernel drivers, `audio_effects.xml`, or vendor HAL configuration. Device-level post-processing may exist when the OEM implements and exposes the route, but the Android 11 device-effect route is not a universal public API for ordinary APKs.

Tidak ada penulisan langsung ke kernel / HAL. APK tidak mengubah firmware, driver kernel, `audio_effects.xml`, atau konfigurasi vendor HAL. Pemrosesan tingkat perangkat dapat tersedia bila OEM mengimplementasikan dan mengekspos route tersebut, tetapi device-effect Android 11 bukan API publik universal untuk APK biasa.

05 Adaptive EQ Backend | Backend Equalizer Adaptif

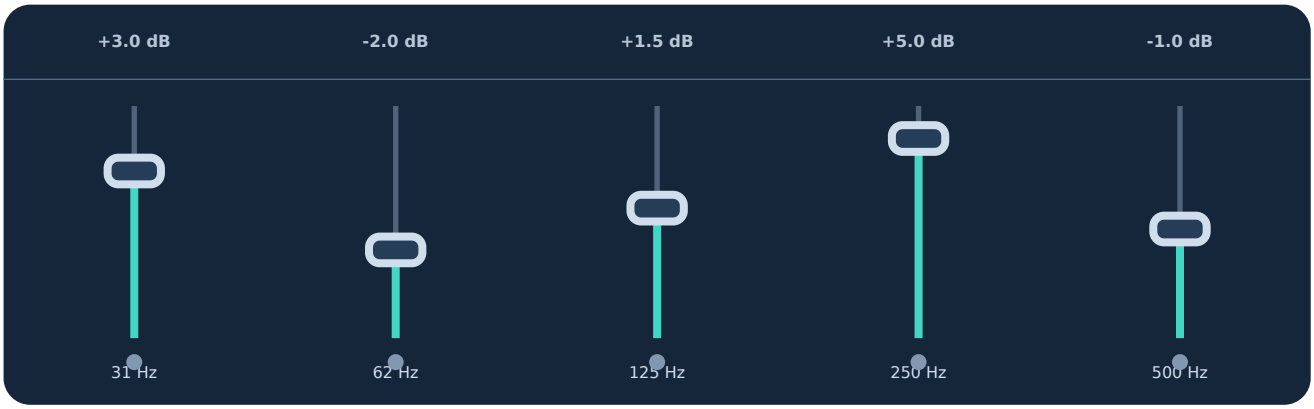
How the app chooses between configurable 10-band DSP and native Equalizer fallback. / Cara aplikasi memilih DSP 10-band atau fallback Equalizer native.

Decision step	Implementation logic	Result
1. API guard	If Android API >= 28, attempt to build a DynamicsProcessing configuration.	API 28+ eligible for the 10-band attempt.
2. Configure Pre-EQ	10 bands; frequency-resolution variant; stereo/two channel configuration; input gain stage present; limiter stage in-use but initially bypassed.	Requested graphic targets: 31, 62, 125, 250, 500, 1k, 2k, 4k, 8k, 16k Hz.
3. Construct and verify	Create `DynamicsProcessing(0, 0, config)`; require `hasControl()` and positive `channelCount`.	Backend = DYNAMICS_PROCESSING_10.
4. Fallback	If DP path fails, create `Equalizer(0, 0)`; require `hasControl()` and `numberOfBands > 0`.	Backend = NATIVE_EQUALIZER with exact device-reported bands.
5. Optional effects	Try `BassBoost(0,0)` and `Virtualizer(0,0)` independently; retain only if control is granted.	Optional feature availability is independent of the EQ backend.

Important frequency mapping detail. The Android `DynamicsProcessing.EqBand` configuration represents upper cutoff frequencies rather than graphic-EQ centre points. The project therefore uses geometric boundary approximations for ten target labels. The UI labels are portable target centres; the actual DSP band filters are configured from the corresponding cutoff boundaries.

Detail pemetaan frekuensi. Konfigurasi `DynamicsProcessing.EqBand` Android merepresentasikan frekuensi batas atas (cutoff), bukan titik tengah graphic EQ. Karena itu project memakai pendekatan batas geometrik untuk sepuluh label target. Label UI adalah target frekuensi portable; filter DSP aktual dikonfigurasi menggunakan batas cutoff yang sesuai.

Target label	Configured upper cutoff (Hz)	Target label	Configured upper cutoff (Hz)
31 Hz	43.8	500 Hz	707.1
62 Hz	88.1	1 kHz	1414.2
125 Hz	176.8	2 kHz	2828.4
250 Hz	353.6	4 kHz	5656.8
8 kHz	11313.7	16 kHz	20000



Band values. The controller reads band levels in millibels from native Equalizer and converts DynamicsProcessing gain values from dB to the app's millibel-oriented model. The DP path clamps the graphic band range to -15.00 dB to +15.00 dB. Native Equalizer uses the device's reported `bandLevelRange` instead of assuming a fixed range.

Nilai band. Controller membaca level native Equalizer dalam millibel dan mengonversi gain DynamicsProcessing dari dB ke model aplikasi berbasis millibel. Jalur DP membatasi rentang graphic band pada -15,00 dB hingga +15,00 dB. Native Equalizer memakai `bandLevelRange` dari perangkat, bukan rentang tetap yang diasumsikan.

06 Processing Controls | Preamp, EQ, Bass, Virtualizer and Limiter

The role of each stage and the values currently defined by the implementation. / Peran setiap stage dan nilai yang ditetapkan implementasi.

The project treats these controls as separate effect stages. They are not a single master-volume slider, and availability is checked independently from the EQ band backend.

Project memperlakukan kontrol ini sebagai stage effect yang terpisah. Fitur-fitur tersebut bukan satu slider master volume, dan ketersediaannya diperiksa terpisah dari backend band EQ.

Control	DSP / API role	Implemented behavior in v3.0.9
Preamp / Input Gain	Input gain stage before Pre-EQ in DynamicsProcessing.	Toggle + slider; range -12 dB to +6 dB; OFF applies 0 dB; read-back from channel 0. Available only with controlled DynamicsProcessing backend.
Graphic EQ	Pre-EQ stage with 10 configured bands, or native Equalizer bands.	Each fader maps to a band and applies per-channel value for DP. Native fallback uses reported band indices and supported limits.
BassBoost	Low-frequency effect (`BassBoost` strength API 0-1000).	Optional session-0 instance; checks `hasControl`, `strengthSupported`, and reads rounded strength. Toggle + strength slider.
Virtualizer	Spatial/perceived channel effect, not bass gain.	Optional session-0 instance; checks control/strength support; API is deprecated from API 35, so future spatializer path needs separate detection.
Limiter	DynamicsProcessing dynamics stage near the end of the processing chain.	Stage configured in-use, initially disabled; ON/OFF. Default parameters in source: threshold -1 dBFS, ratio 20:1, attack 5 ms, release 100 ms, post-gain 0 dB.

LIMITER IS DYNAMIC, NOT A STATIC GAIN KNOB

The limiter continuously responds when the signal crosses its threshold. It does not raise or lower all audio by a fixed percentage. Its job is to reduce peaks above the threshold according to ratio/attack/release. The current UI exposes ON/OFF while the DSP parameters remain fixed in code. / Limiter bereaksi secara dinamis ketika sinyal melewati threshold. Ia bukan pengatur gain tetap dengan persentase. Tugasnya mengurangi puncak di atas threshold sesuai ratio/attack/release. UI saat ini menyediakan ON/OFF, sedangkan parameter DSP ditetapkan di source.

Headroom and clipping. EQ boosts can raise signal peaks. A negative preamp can create more headroom before EQ; the limiter can control peaks later in the chain. Neither guarantees immunity from clipping in downstream hardware, DAC, amplifier, Bluetooth codec, or vendor processing.

Headroom dan clipping. Boost EQ dapat menaikkan puncak sinyal. Preamp negatif dapat menyediakan ruang sebelum EQ; limiter mengendalikan puncak pada tahap berikutnya. Keduanya tidak menjamin bebas clipping pada hardware lanjutan seperti DAC, amplifier, codec Bluetooth, atau pemrosesan vendor.

Virtualizer note. Android marks the classic Virtualizer API deprecated on API 35+. The project currently uses the legacy `Virtualizer` class where available; Spatializer is not treated as a drop-in replacement for bass enhancement or graphic EQ.

Catatan Virtualizer. Android menandai API Virtualizer klasik sebagai deprecated pada API 35+. Project saat ini memakai class `Virtualizer` lama bila tersedia; Spatializer tidak dianggap pengganti langsung Bass Boost atau graphic EQ.

07 Reverb Capability vs Player Attachment | Deteksi Reverb dan Koneksi Player

Why a detected descriptor is not the same as an effect processing VLC or other apps. / Mengapa descriptor yang terdeteksi tidak sama dengan pemrosesan VLC atau aplikasi lain.

What is implemented. `EffectCapabilityProbe` calls `AudioEffect.queryEffects()`, maps each descriptor's name, implementor, type UUID, implementation UUID and connect mode, then filters descriptors whose names contain “reverb”. It separately identifies auxiliary and post-processing connection modes. `EqualizerActivity` displays the device/API identity and a reasoned disabled state for external-player Reverb.

Yang diimplementasikan. `EffectCapabilityProbe` memanggil `AudioEffect.queryEffects()`, memetakan nama descriptor, implementor, type UUID, implementation UUID, dan connect mode, lalu menyaring descriptor yang namanya mengandung “reverb”. Mode auxiliary dan post-processing diidentifikasi terpisah. `EqualizerActivity` menampilkan identitas perangkat/API serta alasan status disabled untuk Reverb player eksternal.

Step	Auxiliary Reverb contract	Project v3.0.9 status
1. Discover	Find the Reverb descriptor and its connect mode.	Implemented: read-only descriptor scan.
2. Create / own effect	Create the auxiliary effect and obtain a valid effect ID / handle.	Not implemented for external player path.
3. Attach player	Compatible player must call `attachAuxEffect(effectId)` on its AudioTrack / MediaPlayer output.	Not available: Floating DJ does not own VLC or another third-party player's track.
4. Set send level	Player must call `setAuxEffectSendLevel(level)` with a non-zero value to send signal to the effect.	Not available: no third-party send-level control in this app.
5. Mix wet/dry output	The player/output chain must route the effect return with its original (dry) signal.	Not verified for other apps / global output.

VLC and other players. Integration is possible only if the player exposes a compatible audio session or an integration point that permits the external app to attach an effect and set send level. The EQ module does not currently discover a third-party player's session ID, inject an effect into VLC, or modify a player-specific audio pipeline.

VLC dan player lain. Integrasi hanya mungkin jika player menyediakan audio session yang kompatibel atau titik integrasi yang memungkinkan aplikasi eksternal memasang effect dan mengatur send level. Modul EQ saat ini tidak menemukan session ID player pihak ketiga, tidak menyisipkan effect ke VLC, dan tidak mengubah pipeline audio khusus player.

WHY THE SWITCH IS DISABLED / MENGAPA TOGGLE DISABLED

A detected NXP Software Reverb descriptor confirms that the firmware advertises a Reverb implementation; it does not identify a player session that the app can control. The disabled control prevents a false promise of global Reverb. / Descriptor Reverb NXP Software menunjukkan firmware mengiklankan implementasi Reverb; descriptor itu tidak memberikan session player yang dapat dikendalikan aplikasi. Kontrol disabled mencegah klaim palsu bahwa Reverb global sudah aktif.

The screenshot supplied during device testing reported Xiaomi 220333QAG / Android 11 (API 30) and an Auxiliary Reverb descriptor attributed to NXP Software Ltd. This is a device-observed descriptor, not a measurement of Reverb output on VLC.

Screenshot pengujian perangkat melaporkan Xiaomi 220333QAG / Android 11 (API 30) dan descriptor Auxiliary Reverb dengan implementor NXP Software Ltd. Ini adalah descriptor yang teramati pada perangkat, bukan pengukuran output Reverb pada VLC.

08 Engine Ownership & Lifecycle | Kepemilikan Engine dan Lifecycle

How EQ remains separate from the visualizer Activity and how state is restored. / Bagaimana EQ terpisah dari Activity visualizer dan bagaimana state dipulihkan.

Event	Owner / action	Expected behavior
User opens EQ screen	MainActivity opens `EqualizerActivity`.	The EQ UI reads Theme Studio preferences and loads the saved Current State.
User switches EQ ON	Activity calls `EqEngineController.activate()`, persists `engine_enabled=true`, and starts `EqEngineService` as a foreground service on supported Android versions.	Controller chooses DynamicsProcessing or native Equalizer and validates control.
Effect created	`EqEngineController` owns native handles and control listeners.	UI reads backend name, active state, band count, route inventory and capability values.
Slider / switch edit	Activity calls controller setters; controller applies parameters and reads back actual values.	Values update the in-memory profile and are persisted as Current State.
Activity closes / pauses	Activity captures current values and persists the working profile; it does not release long-lived native handles.	Service remains lifecycle owner while enabled.
Service restarts	EqEngineService checks `engine_enabled`, activates backend, sets enabled, then loads Current State and reapplies bands/effects.	Best-effort restoration of stored settings.
EQ OFF / service stop	Service disables effects, releases handles, stops foreground mode and refreshes the visualizer notification.	Native resources are released; status should return OFF.

Control-loss handling. The controller registers `AudioEffect.OnControlStatusChangeListener` for the main EQ backend and optional BassBoost/Virtualizer instances. When another effect gains control, the app records an error/status message. A UI toggle or persisted preference is not sufficient to prove continued control; `hasControl()` and effect read-back are the relevant checks.

Penanganan kehilangan kontrol. Controller mendaftarkan `AudioEffect.OnControlStatusChangeListener` untuk backend EQ utama dan instance opsional BassBoost/Virtualizer. Jika effect lain mengambil kontrol, aplikasi mencatat pesan error/status. Toggle UI atau preferensi tersimpan tidak cukup membuktikan kontrol tetap dimiliki; pemeriksaan yang relevan adalah `hasControl()` dan pembacaan kembali parameter effect.

BACKGROUND SERVICE IS NOT A GUARANTEE AGAINST OS KILLING

A foreground service improves user-visible lifecycle handling but does not make an app immune to Android process termination, OEM battery management, user force-stop, permission revocation, or service restrictions. The implementation restores from persisted state when the service is restarted, but device behavior must be tested. / Foreground service membantu pengelolaan lifecycle yang terlihat oleh pengguna, tetapi tidak membuat aplikasi kebal dari penghentian proses Android, manajemen baterai OEM, force-stop pengguna, pencabutan izin, atau pembatasan service. Implementasi mencoba memulihkan state saat service berjalan kembali, tetapi perilaku perangkat tetap harus diuji.

Visualizer independence. `FloatingVisualizerService` retains visualization/overlay responsibilities. EQ uses its own `EqEngineService`; the main visualizer renderer and EQ controller are not merged into one audio object. The final project diff from v3.0.0 to v3.0.9 shows extensive EQ-specific growth while the visualizer service remains a separate service.

Independensi Visualizer. `FloatingVisualizerService` tetap bertanggung jawab atas visualisasi dan overlay. EQ menggunakan `EqEngineService` tersendiri; renderer visualizer utama dan controller EQ tidak digabung menjadi satu objek audio. Perbandingan project v3.0.0 dan v3.0.9 menunjukkan penambahan besar pada modul EQ, sementara service visualizer tetap terpisah.

09 Preset Model & Data Portability | Model Preset dan Portabilitas Data

Current State, saved presets, schema, and mapping between different band counts. / Current State, preset tersimpan, schema, dan pemetaan antarjumlah band.

`EqProfile` stores an ID/name, frequency centres, gain values, BassBoost enable/strength, Virtualizer enable/strength, Limiter enable, Preamp enable and Preamp dB value. `EqPresetRepository` persists named items and the working Current State in SharedPreferences under the `eq_presets_v1` store.

`EqProfile` menyimpan ID/nama, frekuensi pusat, nilai gain, status/strength BassBoost, status/strength Virtualizer, status Limiter, status Preamp, dan nilai Preamp dB. `EqPresetRepository` menyimpan item bernama dan Current State dalam SharedPreferences bernama `eq_presets_v1`.

JSON field	Meaning	Portability notes
schemaVersion	Portable format version (currently 1).	Rejects unsupported schema versions.
presetId / name	Profile identity and display name.	Imported profile receives a new local ID.
eq.centersHz	Target band frequency array.	Stores frequency intent rather than only vendor band index.
eq.gainDb	Gain per target frequency.	Array length must match centre frequency array.
preamp.enabled / gainDb	Input gain setting and state.	Applies only when active backend supports Input Gain.
bassBoost / virtualizer	Enabled flag and normalized strength.	Device capability and read-back can differ.
limiter.enabled	Limiter state.	Only supported when a controlled DynamicsProcessing stage is available.

Cross-device mapping. During profile application, the app compares stored frequency points with the active band's centre frequency. Exact matches are preferred. If the target engine has a different band layout, the Activity uses logarithmic-frequency interpolation between adjacent stored points; values are clamped to the active engine's valid range. The portable source profile is not silently rewritten just because the target device exposes fewer bands.

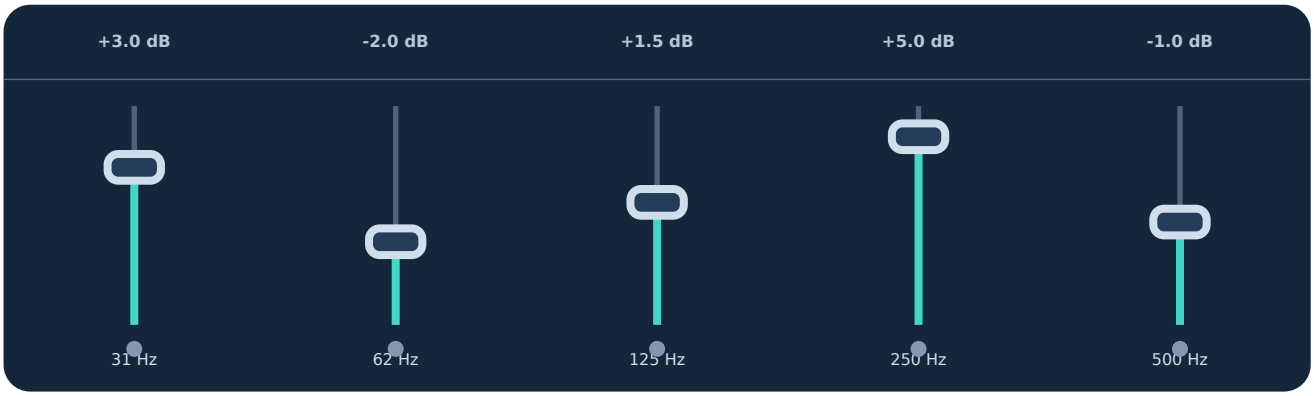
Pemetaan lintas perangkat. Saat preset diterapkan, aplikasi membandingkan titik frekuensi tersimpan dengan frekuensi pusat band aktif. Kecocokan persis diprioritaskan. Jika susunan band engine target berbeda, Activity menggunakan interpolasi frekuensi logaritmik di antara titik yang berdekatan; nilai dibatasi ke rentang engine aktif. Profile sumber tidak diam-diam ditulis ulang hanya karena perangkat tujuan menyediakan band lebih sedikit.

STARTER PRESETS / PRESET AWAL

5-band: Flat, Bass, Vocal, Treble. 10-band: Flat, Bass, Vocal, Dance, Acoustic. These are starter curves, not calibrated profiles for every speaker/headset. The repository inserts missing built-ins without replacing existing user presets. / 5-band: Flat, Bass, Vocal, Treble. 10-band: Flat, Bass, Vocal, Dance, Acoustic. Kurva ini adalah preset awal, bukan kalibrasi untuk setiap speaker/headset. Repository menambahkan preset bawaan yang belum ada tanpa menimpa preset pengguna.

10 User Interface & Notification Integration | Integrasi UI dan Notifikasi

A separate EQ workspace that follows the main Theme Studio. / Workspace EQ terpisah yang mengikuti Theme Studio utama.



Theme Studio link. `EqualizerActivity` reads the same preference namespace used by `MainActivity`: `theme_mode`, `theme_accent`, `theme_surface_color`, `theme_edge_color`, text colour keys, `app_language`, and `ui_scale_percent`. The Activity applies theme-sensitive panel gradients, typography, controls, status bars and WindowInsets.

Integrasi Theme Studio. `EqualizerActivity` membaca preferensi yang sama dengan `MainActivity`: `theme_mode`, `theme_accent`, `theme_surface_color`, `theme_edge_color`, key warna teks, `app_language`, dan `ui_scale_percent`. Activity menerapkan gradasi panel sesuai tema, tipografi, kontrol, system bar, dan WindowInsets.

UI area	v3.0.9 behavior
Main-page EQ card	Opens EqualizerActivity; displays green ACTIVE or red OFF badge based on persisted request plus runtime controller active state.
EQ header / master switch	Shows experimental output-mix context and an EQ active/off badge.
Band canvas	5-band layout can be centred; 10-band layout scrolls horizontally; custom MixerFaderView supports vertical gain adjustment and directional gesture handling.
Band value input	Tap dB value to open direct editing; preview updates fader, then commit/cancel/navigation action decides whether to persist.
Preset UI	Current State vs saved preset status; create/save-to/rename/delete/flat; import/export chooser.
Effect cards	Preamp, BassBoost, Virtualizer, Limiter and Reverb are shown with capability-dependent availability and state information.
Notification	FloatingVisualizerService notification includes Floating, Start/Stop Visualizer, EQ Engine and MediaProjection controls where supported.

Notification architecture. The EQ notification toggle is dispatched through a service action and starts/stops `EqEngineService`; the Start/Stop Visualizer row controls the visualizer service's audio engine separately. These controls are not a `MediaSession` transport interface; they are app-owned notification actions.

Arsitektur notifikasi. Toggle EQ dikirim melalui action service untuk memulai/menghentikan `EqEngineService`; baris Start/Stop Visualizer mengendalikan audio engine visualizer secara terpisah. Kontrol ini bukan antarmuka transport `MediaSession`, melainkan action notifikasi milik aplikasi.

11 Android Compatibility & Permissions | Kompatibilitas Android dan Izin

The SDK envelope and the API gates used by the code. / Rentang SDK dan pemeriksaan API yang digunakan kode.

Android / API	Relevant capability	Project handling
API 26+	Project minimum SDK.	Base app, visualizer and classic audio-effect APIs target this floor.
API 28+	`DynamicsProcessing`.	Runtime guard before attempting 10-band Pre-EQ, Input Gain and Limiter stage.
API 29+	`AudioPlaybackCapture` platform capability.	Existing visualizer backup/capture path may use MediaProjection; it is not used as an insert EQ path.
API 30 / Android 11	Vendor/device post-processing route concepts; System API and OEM configuration constraints.	No ordinary APK assumption of device-wide effect attach or kernel access.
API 33+	Notification permission and newer foreground-service behaviors.	POST_NOTIFICATIONS declared; user/system permission and foreground-service restrictions still apply.
API 34+	Edge-to-edge default changes for apps targeting newer APIs; several legacy APIs deprecations.	EqualizerActivity contains system bar inset handling on supported platform versions.
API 35+	Classic Virtualizer deprecated; Spatializer preferred for spatialization capability queries.	Legacy Virtualizer may remain available, but a future Spatializer capability path requires separate implementation.
Target API 36	Current target SDK.	Edge-to-edge behavior must be verified on Android 15+; target SDK is not the same as min SDK.

Declared permissions / Izin yang dideklarasikan. The manifest includes RECORD_AUDIO, POST_NOTIFICATIONS, MODIFY_AUDIO_SETTINGS, SYSTEM_ALERT_WINDOW, FOREGROUND_SERVICE, FOREGROUND_SERVICE_MEDIA_PROJECTION, and FOREGROUND_SERVICE_SPECIAL_USE. The EQ service is declared with `foregroundServiceType="specialUse"` and a subtype property describing user-enabled audio effect control/status.

Manifest memuat RECORD_AUDIO, POST_NOTIFICATIONS, MODIFY_AUDIO_SETTINGS, SYSTEM_ALERT_WINDOW, FOREGROUND_SERVICE, FOREGROUND_SERVICE_MEDIA_PROJECTION, dan FOREGROUND_SERVICE_SPECIAL_USE. EQ service dideklarasikan dengan `foregroundServiceType="specialUse"` dan property subtype yang menjelaskan kontrol/status audio effect yang diaktifkan pengguna.

DEVICE TEST PROFILE / PROFIL PENGUJIAN PERANGKAT

The supplied screenshots identify Xiaomi 220333QAG / Redmi 10C and Android 11 (API 30). The v3.0.9 user test reported that the 10-band DynamicsProcessing sliders audibly affected the tested output. This is a valuable device-specific observation, not a universal compatibility certification. / Screenshot yang diberikan mengidentifikasi Xiaomi 220333QAG / Redmi 10C dan Android 11 (API 30). Pengujian pengguna pada v3.0.9 melaporkan slider DynamicsProcessing 10-band memengaruhi output yang diuji. Ini adalah observasi spesifik perangkat, bukan sertifikasi kompatibilitas universal.

Note: The route monitor calls `AudioManager.getDevices(GET_DEVICES_OUTPUTS)` and lists output device types returned by the system. This is an output-device inventory; it should not be interpreted as proof that each listed route is actively carrying the target app's or another app's stream at that instant.

Catatan: route monitor memanggil `AudioManager.getDevices(GET_DEVICES_OUTPUTS)` dan menampilkan jenis output device yang dikembalikan sistem. Ini adalah inventaris output device; bukan bukti bahwa setiap route yang tercantum sedang membawa stream aplikasi target pada saat itu.

12 Verification & Acceptance Matrix | Matriks Verifikasi dan Kriteria Penerimaan

Separate code inspection, user-observed results and tests still required. / Memisahkan pemeriksaan source, hasil observasi pengguna dan pengujian lanjutan.

Verification item	Evidence available	Status / next test
Version metadata	Final source archive `module.toml`: versionName 3.0.9, versionCode 81; minSdk 26, targetSdk 36, compileSdk 36.	Verified in archive.
Module separation	EQ classes under `eq/core`, `eq/service`, `eq/model`, `eq/data`, `eq/route`; dedicated `EqualizerActivity`.	Verified by source inspection.
10-band backend	Source configures 10-band DynamicsProcessing. User screenshot reports 10 active bands / control=true / ACTIVE and user confirms audible output change.	Device-observed; independent lab measurement not supplied.
Native fallback	Code uses actual native Equalizer band count, center frequencies and range.	Source-verified; additional handset tests required.
Preset persistence	SharedPreferences Current State and named preset JSON; service re-applies stored values on restart.	Source-verified; test app force-stop/restart and OEM battery policies.
Preamp / limiter	Input Gain setter/getter and limiter stage methods; configured defaults visible in source/UI.	Source-verified; acoustic/output measurement still required.
Bass / virtualizer	Optional effects created and checked for control; strength setters use rounded read-back.	Source-verified; route-by-route testing required.
Reverb	Descriptor scan only; UI disabled for external player attachment.	Expected limitation; do not test as global Reverb.
Notification actions	Service action toggles EQ; independent action starts/stops visualizer.	Source-verified; test notification permission, app swipes, process restart, OEM restrictions.
Visual/UI behavior	Theme preferences, custom fader, horizontal band viewport, preset chooser, direct input code.	Source-verified; screen-size, gesture and accessibility testing recommended.

Recommended device test matrix / Matriks pengujian yang disarankan

- Test built-in speaker, wired headset, Bluetooth A2DP and USB audio independently; record exact Android build and route. / Uji speaker internal, headset kabel, Bluetooth A2DP dan USB secara terpisah; catat build Android dan route.
- Compare EQ disabled vs enabled at a comfortable listening level using the same source; do not use loud volume as a test method. / Bandingkan EQ OFF dan ON pada volume nyaman dengan sumber audio sama.
- Move each band and verify read-back values; compare bypass and processed state using a reliable post-effect measurement point if available. / Geser tiap band dan periksa read-back; bandingkan bypass dan proses pada titik ukur pasca-effect yang tepercaya jika tersedia.
- Verify service persistence after closing the EQ screen, returning to MainActivity, screen-off, and normal service recreation. / Periksa persistensi service setelah EQ ditutup, kembali ke MainActivity, layar mati, dan service dibuat kembali.
- Test 5-band devices and 10-band devices separately; import profiles in both directions and inspect interpolation warnings/behavior. / Uji perangkat 5-band dan 10-band terpisah; impor preset dua arah dan periksa pemetaan.

13 Known Limitations & Engineering Roadmap | Batasan dan Arah Pengembangan

A truthful handover for future developers. / Catatan serah-terima yang jujur untuk pengembang berikutnya.

Topic	Current state	Possible future work
System-wide EQ	Session-0 output-mix experiment. Device and route dependent.	Evaluate only supported public/vendor integration routes; do not depend on hidden APIs or direct kernel writes.
Reverb	Descriptor detection; external player attach/send not implemented.	Player-owned integration (if app owns AudioTrack/MediaPlayer), or documented OEM integration. A third-party player must expose a compatible session/extension.
Spatial audio	Legacy Virtualizer where supported; API deprecated on newer Android.	Add separate Spatializer capability query and device/output route handling.
Limiter controls	ON/OFF with fixed source defaults.	Expose safe threshold/attack/release controls only after validating read-back, stability, headroom and audible behavior.
Preset transfer	Schema v1 JSON stores target frequency points/gain and effect state.	Version schema migrations; preserve unknown future fields; show mapping summary when target bands differ.
Output route proof	AudioManager output device inventory and effect control status.	Add stronger route-specific evidence/diagnostics if public API permits; distinguish attached devices from active per-stream route.
Background survival	Foreground service + stored state restoration.	Test OEM battery restrictions; surface user guidance; handle permission/notification denial and service start failure.

Code ownership guidelines / Panduan kepemilikan kode

- Keep UI-only changes in `EqualizerActivity`; avoid mixing visualizer rendering logic into EQ controls.
- Keep native effect creation, control listeners, parameter read-back and release inside `EqEngineController`.
- Keep the foreground lifecycle and restart restoration in `EqEngineService`; do not create duplicate effect handles from Activity and Service concurrently.
- Keep portable schema, validation and persistence in `EqPresetRepository` / `EqProfile`.
- Treat descriptor presence, active control, effect enabled state and audible output as four different evidence levels.

Jangan menulis ulang renderer Visualizer ketika hanya UI EQ yang perlu berubah. Hindari membuat effect handle ganda dari Activity dan Service. Pisahkan empat tingkat bukti: descriptor tersedia, kontrol dimiliki, effect enabled, dan perubahan audio aktual.

RELEASE BASELINE / BASELINE RILIS

V3.0.9 is the final Version 3 baseline documented here. Project metadata in the supplied final CAPROJ archive: versionName 3.0.9, versionCode 81, compileSdk 36, minSdk 26, targetSdk 36. The user confirmed a successful CodeAssist debug build/install for this version after the version-metadata correction. / V3.0.9 adalah baseline akhir Versi 3 dalam laporan ini. Metadata CAPROJ final: versionName 3.0.9, versionCode 81, compileSdk 36, minSdk 26, targetSdk 36. Pengguna mengonfirmasi build/install debug CodeAssist berhasil setelah koreksi metadata versi.

14 References & Source Map | Referensi dan Peta Source

Primary project artifacts and Android documentation references from the original research blueprint. / Artefak project utama dan referensi Android dari blueprint riset awal.

Primary implementation artifacts examined / Artefak implementasi utama yang diperiksa

- `Floating-DJ-Visualizer-V3.0.0.caproj` - original Version 3 modular EQ baseline (versionName 3.0.0, versionCode 70).
- `Floating-DJ-Visualizer-V3.0.9-VersionCode-Fix.caproj` - final V3.0.9 project archive (versionName 3.0.9, versionCode 81).
- `Floating-DJ-Visualizer-V3.0.0-Android-Audio-EQ-Research-Blueprint.pdf` - 10-page platform/audio research and initial implementation plan, dated 12 November 2026.
- Device screenshots supplied during v3 testing - evidence of 10-band status, route inventory and Reverb descriptor on Xiaomi 220333QAG / Android 11.
- User-reported V3.0.9 result - successful CodeAssist debug build/install and audible change when manipulating 10 EQ bands on the test handset.

Android / AOSP references listed in the source research blueprint

Ref	Title / resource	URL
[1]	Android AudioEffect API	https://developer.android.com/reference/android/media/audiofx/AudioEffect
[2]	Android Equalizer API	https://developer.android.com/reference/android/media/audiofx/Equalizer
[3]	Android BassBoost API	https://developer.android.com/reference/android/media/audiofx/BassBoost
[4]	Android DynamicsProcessing API	https://developer.android.com/reference/android/media/audiofx/DynamicsProcessing
[5]	Android Virtualizer API	https://developer.android.com/reference/android/media/audiofx/Virtualizer
[6]	Android PresetReverb API	https://developer.android.com/reference/android/media/audiofx/PresetReverb
[7]	Android EnvironmentalReverb API	https://developer.android.com/reference/android/media/audiofx/EnvironmentalReverb
[8]	AudioManager.generateAudioSessionId	https://developer.android.com/reference/android/media/AudioManager#generateAudioSessionId()
[9]	AOSP - Audio effects	https://source.android.com/docs/core/audio/audio-effects
[10]	AOSP - Audio HAL	https://source.android.com/docs/core/audio/implement
[11]	AOSP - Audio architecture	https://source.android.com/docs/core/audio
[12]	Android LoudnessEnhancer API	https://developer.android.com/reference/android/media/audiofx/LoudnessEnhancer
[13]	AudioPlaybackCaptureConfiguration	https://developer.android.com/reference/android/media/AudioPlaybackCaptureConfiguration
[14]	AOSP API level reference	https://source.android.com/docs/setup/reference/build-numbers
[15]	Google Play target API requirements	https://support.google.com/googleplay/android-developer/answer/11926878
[16]	Xiaomi Redmi 10C official specifications	https://www.mi.co.id/id/product/redmi-10c/specs/
[17]	RootlessJamesDSP - example and stated limitations	https://f-droid.org/en/packages/me.timschneeberger/rootlessjamesdsp/

Methodology / Metodologi. The report is grounded in source inspection of the initial and final CAPROJ archives, the supplied Version 3 research blueprint, project iteration notes in the conversation, and user-provided device observations. API and platform constraints are attributed to the official documentation sources listed above. No claim is made that all Android devices, apps, or output routes behave identically. No independent lab measurement of the handset's post-effect output was supplied.

Laporan ini didasarkan pada pemeriksaan source CAPROJ awal dan final, blueprint riset Versi 3, catatan iterasi project dalam percakapan, serta observasi perangkat dari pengguna. Batas API/platform merujuk dokumentasi resmi pada daftar di atas. Laporan tidak mengklaim semua perangkat, aplikasi, atau output route Android berperilaku sama. Tidak tersedia pengukuran laboratorium independen atas output pasca-effect perangkat.

DOCUMENT CLOSE / PENUTUP. Floating DJ Visualizer v3.0.9 adds a modular, experimental audio-control layer alongside the existing floating visualizer. / V3.0.9 menambahkan lapisan kontrol audio eksperimental yang modular di samping floating visualizer.